05 — Deadlocks in PostgreSQL

Table of Contents

Learning Objectives

After studying this file you will be able to:

- Define a deadlock and explain precisely when one occurs
- Describe PostgreSQL's deadlock detection algorithm and the `deadlock_timeout` parameter
- Parse and interpret the ERROR message PostgreSQL produces when a deadlock is detected
- Identify the three classic deadlock patterns in SQL workloads
- Apply consistent lock-ordering and short-transaction strategies to prevent deadlocks
- Write `pg_locks` queries to investigate a deadlock situation
- Implement correct retry logic with exponential back-off in application code
- Distinguish deadlocks from lock timeouts and statement timeouts

---

What Is a Deadlock?

A **deadlock** occurs when two or more transactions are each waiting for a lock held by another member of the cycle — so none of them can ever make progress.

```
Simplest deadlock: two transactions, two rows

  Transaction A              Transaction B
  -----------------------    -----------------------
  Lock row 1 (granted)       Lock row 2 (granted)
  Attempt row 2 --> WAIT     Attempt row 1 --> WAIT
           ^                          |
           |________ cycle ___________|

Neither A nor B can proceed. This is a deadlock.
```

Formal Definition

A **wait-for graph** (WFG) has one node per active transaction and a directed edge from T1 to T2 when T1 is waiting for a lock held by T2. A deadlock exists when the WFG contains a **directed cycle**.

```
Wait-For Graph (no deadlock):
  A --> B --> C      (A waits for B, B waits for C; C is not waiting)

Wait-For Graph (deadlock):
  A --> B --> C --> A   (cycle detected!)
```

What Resources Can Deadlock?

In PostgreSQL, deadlocks can occur on:

- **Row-level locks** — the most common case (UPDATE/DELETE/SELECT FOR UPDATE)
- **Table-level locks** — e.g., two transactions that both need SHARE and ROW EXCLUSIVE on two different tables in opposite order
- **Advisory locks** — if applications acquire multiple advisory locks in inconsistent order
- **Tuple locks** — rare, but possible in high-contention INSERT scenarios

---

How PostgreSQL Detects Deadlocks

PostgreSQL uses a **timeout-based detection** approach rather than continuous WFG maintenance.

The deadlock_timeout Parameter

```
SHOW deadlock_timeout;  -- default: 1s
```

When a lock request cannot be immediately granted:

1. The backend waits for the lock normally.
2. If `deadlock_timeout` elapses without the lock being granted, the backend runs the deadlock detection algorithm.
3. The algorithm builds the current WFG and checks for cycles.
4. If a cycle is found, PostgreSQL **aborts one victim transaction** (the one whose rollback is cheapest — typically the most recently started one).
5. If no cycle is found, the backend continues waiting.

Why Timeout-Based (Not Immediate)?

Immediate detection would require updating a global WFG on every lock acquisition and release — an expensive operation. Since most lock waits resolve quickly (within milliseconds), it is more efficient to wait `deadlock_timeout` before checking. The default 1 second means only waits that are already unusual trigger the check.

Algorithm Detail

```
On deadlock check for transaction T:
1. Build set S = {T}
2. For each transaction W in S:
```

```
   a. Find all transactions H that W is waiting for (H holds lock W needs)
   b. For each H:
      - If H == T: CYCLE FOUND -> deadlock, abort T
      - If H not in S: add H to S
3. If no cycle found after exhausting S: not a deadlock, continue waiting
```

Tuning deadlock_timeout

```
-- Lower value: detect deadlocks faster, but more CPU for detection checks
ALTER SYSTEM SET deadlock_timeout = '250ms';

-- Higher value: less CPU overhead, but transactions wait longer before detection
ALTER SYSTEM SET deadlock_timeout = '5s';

SELECT pg_reload_conf();
```

For OLTP systems with tight SLAs, reducing `deadlock_timeout` to 250ms–500ms is reasonable. For batch systems, the default 1s is fine.

Reading the Deadlock ERROR Output

When PostgreSQL detects and resolves a deadlock, the victim transaction receives this error:

```
ERROR:  deadlock detected
DETAIL: Process 12345 waits for ShareLock on transaction 67890;
        blocked by process 99999.
        Process 99999 waits for ShareLock on transaction 12345;
        blocked by process 12345.
HINT:   See server log for query details.
```

Anatomy of the Error

```
ERROR: deadlock detected
        ^--- The exception code is 40P01 (SQLState)
             Catch this specific code in application error handling.

DETAIL: Process 12345 waits for ShareLock on transaction 67890;
        blocked by process 99999.
        ^--- PID 12345 (the victim) was waiting for XID 67890.
             XID 67890 is held by process 99999.

        Process 99999 waits for ShareLock on transaction 12345;
        blocked by process 12345.
        ^--- PID 99999 was waiting for XID 12345 (the victim's own XID).
             This confirms the cycle: 12345 <-> 99999.

HINT:   See server log for query details.
        ^--- The PostgreSQL log (not the client) contains the actual queries.
```

Server Log Output (log_lock_waits must be enabled)

```
ALTER SYSTEM SET log_lock_waits = on;
SELECT pg_reload_conf();
```

Log entry example:

```
2024-01-15 10:23:41 UTC [12345]: ERROR: deadlock detected
2024-01-15 10:23:41 UTC [12345]: DETAIL:
  Process 12345 waits for ShareLock on transaction 67890; blocked by process 99999.
  Process 99999 waits for ShareLock on transaction 12345; blocked by process 12345.
2024-01-15 10:23:41 UTC [12345]: CONTEXT:
  while updating tuple (0,42) in relation "accounts"
2024-01-15 10:23:41 UTC [12345]: STATEMENT:
  UPDATE accounts SET balance = balance - 100 WHERE id = 2
```

Three-Transaction Deadlock

```
ERROR:  deadlock detected
DETAIL: Process 1 waits for ShareLock on transaction T2; blocked by process 2.
        Process 2 waits for ShareLock on transaction T3; blocked by process 3.
        Process 3 waits for ShareLock on transaction T1; blocked by process 1.
```

This is a three-way cycle: T1 -> T2 -> T3 -> T1.

Classic Deadlock Scenarios

Scenario 1: Opposite-Order Row Locking (Most Common)

```
-- Session A:                          -- Session B:
BEGIN;                                 BEGIN;

UPDATE accounts                        UPDATE accounts
  SET balance = balance - 100            SET balance = balance + 50
  WHERE id = 1;                          WHERE id = 2;
-- Locks row id=1                      -- Locks row id=2

UPDATE accounts                        UPDATE accounts
  SET balance = balance + 100            SET balance = balance - 50
  WHERE id = 2;                          WHERE id = 1;
-- Waits for row id=2 (held by B)      -- Waits for row id=1 (held by A)

-- DEADLOCK after deadlock_timeout
```

Root cause: Both transactions lock the same two rows but in opposite order.

Fix: Always UPDATE rows in primary key order:

```
-- Both transactions now lock in order: id=1, then id=2
BEGIN;
  -- Lock the lower id first
  UPDATE accounts SET balance = balance - 100 WHERE id = LEAST(1, 2);
  UPDATE accounts SET balance = balance + 100 WHERE id = GREATEST(1, 2);
COMMIT;
```

Scenario 2: Implicit Lock from Foreign Key

```
-- Table structure:
-- orders(id PK, customer_id FK -> customers(id), ...)

-- Session A:                          -- Session B:
BEGIN;                                 BEGIN;

-- A locks customer row 5 (FOR UPDATE)  -- B inserts order for customer 5
UPDATE customers                       INSERT INTO orders
  SET credit = credit - 200              (customer_id, amount)
  WHERE id = 5;                        VALUES (5, 200);
-- Acquires FOR UPDATE on customer 5   -- FK check acquires FOR KEY SHARE on
customer 5
                                       -- WAITS: A holds FOR UPDATE, incompatible
with FOR KEY SHARE?
                                       -- Actually FOR KEY SHARE *is* compatible
with FOR NO KEY UPDATE
                                       -- but NOT with FOR UPDATE. So B waits here.

-- A then tries to insert an order too  -- B tries to update customer 5
INSERT INTO orders                     UPDATE customers SET credit = credit + 100
  (customer_id, amount)                  WHERE id = 5;
  VALUES (5, 100);                     -- Waits for A's FOR UPDATE
-- This INSERT's FK check waits for B's FOR KEY SHARE on customer 5... DEADLOCK
```

Fix: Be aware that `FOR UPDATE` on parent rows conflicts with FK-triggered `FOR KEY SHARE` on child inserts. Use `FOR NO KEY UPDATE` when you are updating non-key columns of the parent.

Scenario 3: Aggregate + Row Lock Race

```
-- Session A selects maximum ID to use as a new ID, then inserts
BEGIN;
  SELECT max(id) + 1 FROM widgets;  -- returns 100
  INSERT INTO widgets (id, name) VALUES (101, 'Gear');

-- Session B does the same concurrently
BEGIN;
  SELECT max(id) + 1 FROM widgets;  -- also returns 100 (A hasn't committed)
  INSERT INTO widgets (id, name) VALUES (101, 'Spring');
```

```
-- Both try to insert id=101 -> unique violation conflict -> potential deadlock
-- if each is waiting on the other's in-progress insert lock
```

Fix: Use `SERIAL` / `BIGSERIAL` or `gen_random_uuid()` for IDs. Never compute max+1 manually.

Scenario 4: Two-Table Deadlock

```
-- Session A:                          -- Session B:
BEGIN;                                  BEGIN;
LOCK TABLE invoices IN SHARE ROW        LOCK TABLE payments IN SHARE ROW
  EXCLUSIVE MODE;                         EXCLUSIVE MODE;
LOCK TABLE payments IN SHARE ROW        LOCK TABLE invoices IN SHARE ROW
  EXCLUSIVE MODE;  -- WAITS for B         EXCLUSIVE MODE;  -- WAITS for A
-- DEADLOCK
```

Fix: Always acquire table locks in alphabetical (or otherwise consistent) order.

Deadlock Prevention Strategies

Strategy 1: Consistent Lock Ordering

The single most effective prevention technique. Always access (lock) resources in the same global order across all transactions.

```
-- RULE: always lock accounts by ascending id
CREATE OR REPLACE FUNCTION transfer_funds(
    p_from_id bigint,
    p_to_id   bigint,
    p_amount  numeric
) RETURNS void AS $$
DECLARE
  v_first_id  bigint := LEAST(p_from_id, p_to_id);
  v_second_id bigint := GREATEST(p_from_id, p_to_id);
BEGIN
  -- Always lock lower id first, regardless of direction of transfer
  PERFORM id FROM accounts WHERE id = v_first_id  FOR UPDATE;
  PERFORM id FROM accounts WHERE id = v_second_id FOR UPDATE;

  UPDATE accounts SET balance = balance - p_amount WHERE id = p_from_id;
  UPDATE accounts SET balance = balance + p_amount WHERE id = p_to_id;
END;
$$ LANGUAGE plpgsql;
```

Strategy 2: Short Transactions

The shorter a transaction, the less time its locks are held and the smaller the window for a deadlock cycle to form.

```
-- BAD: long transaction with multiple round-trips
BEGIN;
```

```
  SELECT * FROM orders WHERE id = $1 FOR UPDATE;
  -- ... application computes discount (takes 500ms) ...
  UPDATE orders SET discount = $2 WHERE id = $1;
  -- ... application calls external API (takes 1s) ...
  UPDATE inventory SET reserved = reserved + 1 WHERE sku = $3;
COMMIT;

-- GOOD: do expensive work before acquiring locks
-- Step 1 (outside transaction): compute discount, call API, get sku
-- Step 2 (short transaction): only the writes
BEGIN;
  SELECT * FROM orders WHERE id = $1 FOR UPDATE;
  UPDATE orders SET discount = $computed_discount WHERE id = $1;
  UPDATE inventory SET reserved = reserved + 1 WHERE sku = $precomputed_sku;
COMMIT;
```

Strategy 3: Acquire All Locks Upfront

If you know you will need multiple locks, acquire them all at the start of the transaction in a consistent order.

```
BEGIN;
  -- Acquire all needed row locks in one statement, ordered by id
  SELECT id FROM accounts
  WHERE id IN (1, 2, 7, 15)
  ORDER BY id
  FOR UPDATE;

  -- Now do all the updates without risking deadlock
  UPDATE accounts SET balance = balance - 100 WHERE id = 1;
  UPDATE accounts SET balance = balance + 100 WHERE id = 2;
  -- ... other updates ...
COMMIT;
```

Strategy 4: Use Advisory Locks for Global Ordering

```
-- Instead of locking individual rows in inconsistent order,
-- use a single advisory lock per logical operation
BEGIN;
  -- Acquire a single advisory lock that serializes all fund transfers
  PERFORM pg_advisory_xact_lock(hashtext('account_transfer'));

  -- Now no deadlock is possible on transfers
  UPDATE accounts SET balance = balance - 100 WHERE id = p_from;
  UPDATE accounts SET balance = balance + 100 WHERE id = p_to;
COMMIT;
-- Downside: serializes ALL transfers, even on different accounts
```

Strategy 5: Use SELECT FOR UPDATE Early

Acquire the lock at SELECT time rather than at UPDATE time to shorten the "change of lock mode" window.

```
-- BAD: lock upgrade risk
BEGIN;
  SELECT * FROM tasks WHERE id = 42;  -- no lock acquired
  -- ... some time passes ...
  UPDATE tasks SET status = 'done' WHERE id = 42;  -- lock acquired here, potential
race
COMMIT;

-- GOOD: lock at read time
BEGIN;
  SELECT * FROM tasks WHERE id = 42 FOR UPDATE;  -- lock acquired immediately
  UPDATE tasks SET status = 'done' WHERE id = 42;
COMMIT;
```

---

pg_locks Deadlock Investigation Queries

Since a deadlock is resolved in milliseconds by PostgreSQL, you cannot observe it in `pg_locks` after the fact. Instead, use these queries to **diagnose conditions that lead to deadlocks** (circular waits before the deadlock is detected):

Query 1: Find Circular Wait Candidates

```
-- Find transactions waiting for each other (potential deadlock in progress)
WITH lock_graph AS (
    SELECT
        blocked.pid   AS blocked_pid,
        blocker.pid   AS blocker_pid
    FROM pg_stat_activity blocked
    JOIN pg_stat_activity blocker
        ON blocker.pid = ANY(pg_blocking_pids(blocked.pid))
)
SELECT
    a.blocked_pid,
    a.blocker_pid,
    b.blocked_pid AS blocker_also_blocked_by,
    b.blocker_pid AS ultimate_blocker
FROM lock_graph a
JOIN lock_graph b ON b.blocked_pid = a.blocker_pid
WHERE b.blocker_pid = a.blocked_pid;  -- cycle: A waits for B, B waits for A
```

Query 2: Full Lock Chain with Queries

```
-- Show full blocking chain, useful just before a deadlock resolves
SELECT
    blocked.pid                             AS blocked_pid,
    blocked.query                           AS blocked_query,
    blocked.query_start                     AS blocked_since,
    blocker.pid                             AS blocker_pid,
```

```
    blocker.query                           AS blocker_query,
    blocker.xact_start                      AS blocker_txn_start,
    now() - blocker.xact_start              AS blocker_txn_age,
    locks_waiting.mode                      AS waiting_for_mode,
    locks_held.mode                         AS held_mode,
    locks_held.relation::regclass           AS locked_table
FROM pg_stat_activity blocked
JOIN pg_stat_activity blocker
    ON blocker.pid = ANY(pg_blocking_pids(blocked.pid))
JOIN pg_locks locks_waiting
    ON locks_waiting.pid = blocked.pid AND NOT locks_waiting.granted
JOIN pg_locks locks_held
    ON locks_held.pid = blocker.pid
    AND locks_held.granted
    AND locks_held.locktype = locks_waiting.locktype
    AND locks_held.relation IS NOT DISTINCT FROM locks_waiting.relation
ORDER BY blocked.query_start;
```

Query 3: Row-Level Lock Contention

```
-- Find tuple-level locks (row locks specifically)
SELECT
    l.pid,
    l.relation::regclass AS table_name,
    l.page,
    l.tuple,
    l.mode,
    l.granted,
    a.query,
    now() - a.xact_start AS txn_age
FROM pg_locks l
JOIN pg_stat_activity a ON a.pid = l.pid
WHERE l.locktype = 'tuple'
ORDER BY l.granted, txn_age DESC;
```

Query 4: Post-Mortem from pg_log

```
-- Query the PostgreSQL log table if log_destination includes csvlog
-- (Requires pg_log table set up via file_fdw or similar)

-- Or use pg_read_file to scan recent log for deadlock messages:
SELECT * FROM pg_read_file('pg_log/postgresql-2024-01-15_000000.csv')
-- Look for "deadlock detected" strings
```

Query 5: Deadlock Frequency Counter

```
-- Check how often deadlocks occur in the database
SELECT datname, deadlocks, conflicts,
```

```
       blks_hit, blks_read,
       tup_returned, tup_fetched
FROM pg_stat_database
WHERE datname = current_database();

-- Reset stats to measure a specific time window
SELECT pg_stat_reset();
-- ... run workload ...
SELECT datname, deadlocks FROM pg_stat_database WHERE datname = current_database();
```

Application-Level Deadlock Handling

The Right Exception to Catch

PostgreSQL uses **SQLSTATE 40P01** for deadlock errors. Always catch this specific code, not a generic database error.

```
# Python (psycopg2 / psycopg3)
import psycopg2
from psycopg2 import errors
import time, random

def transfer_with_retry(conn, from_id, to_id, amount, max_retries=5):
    for attempt in range(max_retries):
        try:
            with conn.transaction():
                cur = conn.cursor()
                cur.execute(
                    "UPDATE accounts SET balance = balance - %s WHERE id = %s",
                    (amount, from_id)
                )
                cur.execute(
                    "UPDATE accounts SET balance = balance + %s WHERE id = %s",
                    (amount, to_id)
                )
                return  # success
        except errors.DeadlockDetected:
            if attempt == max_retries - 1:
                raise
            # Exponential back-off with jitter
            sleep_time = (2 ** attempt) * 0.1 + random.uniform(0, 0.05)
            time.sleep(sleep_time)
```

```
// Java (JDBC) - SQLSTATE 40P01
int maxRetries = 5;
for (int attempt = 0; attempt < maxRetries; attempt++) {
    try (Connection conn = dataSource.getConnection()) {
        conn.setAutoCommit(false);
        try {
            // ... execute statements ...
```

```
            conn.commit();
            break; // success
        } catch (SQLException e) {
            conn.rollback();
            if ("40P01".equals(e.getSQLState()) && attempt < maxRetries - 1) {
                long backoff = (long)(Math.pow(2, attempt) * 100)
                             + ThreadLocalRandom.current().nextLong(50);
                Thread.sleep(backoff);
            } else {
                throw e;
            }
        }
    }
}
```

```
// Go (pgx)
import "github.com/jackc/pgx/v5/pgconn"

func transferWithRetry(pool *pgxpool.Pool, fromID, toID int, amount float64) error {
    const maxRetries = 5
    for attempt := 0; attempt < maxRetries; attempt++ {
        err := doTransfer(pool, fromID, toID, amount)
        if err == nil {
            return nil
        }
        var pgErr *pgconn.PgError
        if errors.As(err, &pgErr) && pgErr.Code == "40P01" && attempt < maxRetries-1
{
            backoff := time.Duration(math.Pow(2,
float64(attempt))*100)*time.Millisecond +
                       time.Duration(rand.Intn(50))*time.Millisecond
            time.Sleep(backoff)
            continue
        }
        return err
    }
    return fmt.Errorf("deadlock not resolved after %d retries", maxRetries)
}
```

Retry Logic Design Principles

```
1. ROLLBACK immediately on deadlock detection.
   The entire transaction is already aborted — do not attempt any more SQL.

2. RETRY from the beginning of the transaction logic.
   Re-read all data; do not reuse values from before the deadlock.

3. Use EXPONENTIAL BACK-OFF with random jitter.
   Pure fixed-delay retries can cause synchronized retry storms.
```

```
4. Cap the number of retries.
   Three to five retries is sufficient; if deadlocks persist after that,
   fix the lock ordering instead.

5. LOG deadlock occurrences with context.
   Track which code path and which row IDs were involved to identify
   systemic lock ordering problems.

6. Alert on high deadlock rates.
   SELECT deadlocks FROM pg_stat_database should be close to zero.
   Rising deadlock counts indicate an application design problem.
```

PL/pgSQL Retry Loop

```
CREATE OR REPLACE FUNCTION transfer_retry(
    p_from bigint, p_to bigint, p_amount numeric
) RETURNS void AS $$
DECLARE
  v_retries int := 0;
  v_max_retries constant int := 5;
BEGIN
  LOOP
    BEGIN
      UPDATE accounts SET balance = balance - p_amount WHERE id = LEAST(p_from,
p_to);
      UPDATE accounts SET balance = balance + p_amount WHERE id = GREATEST(p_from,
p_to);
      -- If we reach here, success
      EXIT;
    EXCEPTION WHEN deadlock_detected THEN
      v_retries := v_retries + 1;
      IF v_retries >= v_max_retries THEN
        RAISE EXCEPTION 'Transfer failed after % deadlock retries', v_max_retries;
      END IF;
      -- Brief delay (in a real scenario, use pg_sleep with exponential back-off)
      PERFORM pg_sleep(0.1 * v_retries);
    END;
  END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Deadlock vs Lock Timeout vs Statement Timeout

These three mechanisms are often confused:

```
Mechanism         | SQLState | Trigger                                | Victim
------------------+----------+----------------------------------------+-----------------
----------
deadlock_detected | 40P01    | Circular wait cycle confirmed          | One transaction
in cycle
```

```
lock_timeout      | 55P03    | Lock wait exceeds lock_timeout       | The waiting
transaction
statement_timeout | 57014    | Statement runs longer than timeout   | The running
statement
```

How They Interact

```
SET lock_timeout = '2s';       -- If I wait > 2s for ANY lock, my statement errors
SET deadlock_timeout = '500ms'; -- After 500ms of waiting, check for cycles

-- Scenario: lock_timeout fires first (2s) if deadlock takes longer than 2s to
detect
-- Scenario: deadlock detected at 500ms before lock_timeout at 2s fires
```

Lock timeout is proactive (fail my own request early). Deadlock detection is reactive (detect and abort one party in a cycle).

```
-- Check current timeout settings
SHOW deadlock_timeout;
SHOW lock_timeout;
SHOW statement_timeout;
```

Common Mistakes

1. Not Retrying After Deadlock

```
# BAD: letting the deadlock propagate up as an uncaught exception
conn.execute("UPDATE accounts SET balance = balance - 100 WHERE id = 1")
conn.execute("UPDATE accounts SET balance = balance + 100 WHERE id = 2")
conn.commit()  # May raise DeadlockDetected, no retry

# GOOD: wrap in retry loop (see Application-Level Deadlock Handling above)
```

2. Retrying Without Full Re-Read

```
# BAD: reusing read data after a deadlock
balance = conn.execute("SELECT balance FROM accounts WHERE id = 1").fetchone()[0]
# ... deadlock occurred somewhere ...
# balance variable is stale — re-execute the SELECT inside the retry loop!
```

3. Catching Generic Exception Instead of DeadlockDetected

```
# BAD: catches everything, including non-retryable errors
try:
    do_transfer()
except Exception:
```

```
    time.sleep(0.1)
    do_transfer()  # retry even on non-deadlock errors!

# GOOD: catch only SQLSTATE 40P01
except psycopg2.errors.DeadlockDetected:
    time.sleep(0.1)
    do_transfer()
```

4. Deadlock in Batch Updates (Not Ordering by PK)

```
-- BAD: iterates in random/application order
FOR account_id IN (7, 2, 15, 1, 9) LOOP
  UPDATE accounts SET balance = balance + bonus WHERE id = account_id;
END LOOP;

-- GOOD: sort IDs ascending before locking
-- (Or use a single UPDATE with array/ANY to let PostgreSQL handle order)
UPDATE accounts
SET balance = balance + bonus
WHERE id = ANY(ARRAY[1, 2, 7, 9, 15]);  -- PostgreSQL scans in index order
```

5. Long Think-Time Transactions

```
-- BAD: hold row lock while waiting for user confirmation
BEGIN;
  SELECT * FROM cart WHERE session_id = $1 FOR UPDATE;
  -- Wait for "Confirm Purchase" button click (could be minutes)
  UPDATE orders ...;
COMMIT;
-- Creates huge deadlock and contention risk
```

Best Practices

1. **Always lock resources in a consistent global order** — this is the only technique that completely eliminates a class of deadlocks.
2. **Keep transactions as short as possible** — do expensive computation before BEGIN.
3. **Always implement retry logic** for SQLSTATE 40P01 in application code.
4. **Use exponential back-off with jitter** in retry loops to avoid retry storms.
5. **Enable `log_lock_waits`** in development and staging to surface lock ordering issues.
6. **Monitor `pg_stat_database.deadlocks`** — a non-zero and growing count means application code has a lock ordering problem.
7. **Lower `deadlock_timeout` in production OLTP** (250ms–500ms) so deadlocks are resolved faster.
8. **Prefer bulk DML in PK order** over row-by-row processing to avoid deadlocks in batch jobs.
9. **Avoid holding locks across network calls or user interactions.**
10. **Use `FOR UPDATE OF table_name`** when joining multiple tables to avoid accidentally locking rows you do not intend to modify.

Performance Considerations

Cost of Deadlock Detection

Each deadlock detection run traverses the wait-for graph. With N waiting transactions, this is O(N) in the number of edges. At low concurrency (< 100 active transactions), the cost is negligible. At high concurrency, lowering `deadlock_timeout` increases the frequency of these checks.

Transaction Abort Cost

The victim transaction must roll back all its changes. If the transaction had done significant work (large INSERT, complex UPDATE), this rollback can itself be expensive. Short transactions limit the rollback cost.

Deadlock Frequency and Throughput

A single deadlock is not a performance problem. Frequent deadlocks (visible as high `pg_stat_database.deadlocks`) indicate a systemic lock ordering issue that wastes work and reduces effective throughput.

```
-- Monitor deadlock rate over time
SELECT
    datname,
    deadlocks,
    extract(epoch from now() - stats_reset) AS seconds_since_reset,
    round(deadlocks / extract(epoch from now() - stats_reset), 4) AS
deadlocks_per_second
FROM pg_stat_database
WHERE datname = current_database();
```

Impact on pg_locks

A deadlock situation transiently increases entries in `pg_locks` (multiple transactions waiting). After detection, the victim is aborted and its lock entries are removed. There is no long-term impact on pg_locks from deadlocks.

Interview Questions and Answers

Q1. What is a deadlock and what is the minimum number of transactions required?

A: A deadlock is a situation where two or more transactions are each waiting for a lock held by another transaction in the set, creating a circular dependency from which none can escape without external intervention. The minimum is two transactions: A holds lock 1 and waits for lock 2, while B holds lock 2 and waits for lock 1.

Q2. How does PostgreSQL detect deadlocks? Why does it wait before checking?

A: PostgreSQL uses a timeout-based approach. When a transaction waits longer than `deadlock_timeout` (default 1s) for a lock, it runs the deadlock detection algorithm — building the wait-for graph and looking for cycles. Waiting first is an optimization: most lock waits resolve in milliseconds, so immediate detection would waste CPU on graph traversal for the common non-deadlock case.

Q3. Which transaction does PostgreSQL choose as the deadlock victim?

A: PostgreSQL aborts the transaction that ran the deadlock check — typically the one that has been waiting longer than `deadlock_timeout`. This is not necessarily the "youngest" or "smallest" transaction; it is the one whose wait triggered the deadlock detection algorithm.

Q4. What is the SQLSTATE code for a deadlock error and why does it matter?

A: SQLSTATE `40P01`. Applications should catch this specific code to distinguish deadlocks from other errors. The error class `40` is "transaction rollback" (the entire transaction is aborted), which means the application must restart the entire transaction from the beginning, not just retry the failed statement.

Q5. What is the single most effective strategy for preventing deadlocks?

A: Consistent lock ordering — always acquire locks on multiple resources in the same global order (e.g., by ascending primary key) across all code paths. When all transactions agree on the acquisition order, circular waits become impossible because a later transaction always waits behind an earlier one in the same sequence.

Q6. Can you observe a deadlock in pg_locks?

A: Not after the fact — PostgreSQL resolves deadlocks within milliseconds, so by the time you query `pg_locks`, the victim has been rolled back and its entries removed. You can observe the pre-deadlock condition (circular waits) if you query `pg_locks` at exactly the right moment. Use `pg_stat_database.deadlocks` to track historical counts, and `log_lock_waits = on` to capture deadlock details in the server log.

Q7. How does a retry loop for deadlocks differ from a retry loop for lock timeouts?

A: Both require catching the error, rolling back, and retrying. For deadlocks (40P01), the transaction has already been fully rolled back by PostgreSQL — the application just needs to restart it. For lock timeouts (55P03), the statement failed but the transaction may still be partially active (if inside a BEGIN block), so the application must explicitly ROLLBACK before retrying.

Q8. What is the relationship between `deadlock_timeout` and `lock_timeout`?

A: They are independent parameters. `deadlock_timeout` is the time a backend waits before running deadlock detection. `lock_timeout` is the maximum time any lock wait is allowed before the statement is cancelled. If `lock_timeout < deadlock_timeout`, a deadlock may be resolved by a lock timeout (55P03) before the deadlock detection algorithm ever runs (40P01).

Q9. Why is "not retrying with re-read" a dangerous retry mistake?

A: After a deadlock, the entire transaction was rolled back. Any data read before the deadlock is now potentially stale — other committed transactions may have changed it. Reusing stale read values in the retry violates the correctness guarantee that transactions operate on consistent data. The correct approach is to restart from the very first SELECT in the transaction.

Q10. How can you reduce deadlock frequency in a batch job that updates many rows?

A: Sort the rows by primary key before processing them. If all batch jobs process rows in ascending PK order, their lock acquisition sequences are aligned, eliminating circular waits. Alternatively, use a single `UPDATE ... WHERE id = ANY(sorted_array)` which PostgreSQL typically processes in index order.

Q11. What does `log_lock_waits` log and how do you interpret it?

A: When enabled, PostgreSQL logs any lock wait that exceeds `deadlock_timeout` duration. The log includes the waiting PID, the type and object of the lock being waited for, and the holding PID. This is the primary tool

for diagnosing lock ordering problems in development/staging before they become production incidents.

Q12. Can foreign key constraints cause deadlocks?

A: Yes. When a child row is inserted or updated, PostgreSQL acquires `FOR KEY SHARE` on the referenced parent row to prevent it from being deleted. If another transaction holds `FOR UPDATE` on that parent row (e.g., an UPDATE to the parent), the FK check waits. If that other transaction also tries to insert a child row that triggers FK checks on rows locked by the first transaction, a deadlock can occur. Prevention: use `FOR NO KEY UPDATE` when updating non-key parent columns.

Exercises and Solutions

Exercise 1 — Reproduce a Deadlock

```
-- Setup
CREATE TABLE dl_test (id int PRIMARY KEY, val int);
INSERT INTO dl_test VALUES (1, 10), (2, 20);

-- Session A (run first):
BEGIN;
UPDATE dl_test SET val = val + 1 WHERE id = 1;
-- Do NOT commit. Now run Session B.

-- Session B:
BEGIN;
UPDATE dl_test SET val = val + 1 WHERE id = 2;  -- succeeds
UPDATE dl_test SET val = val + 1 WHERE id = 1;  -- blocks (Session A holds id=1)

-- Session A (while Session B is blocked):
UPDATE dl_test SET val = val + 1 WHERE id = 2;
-- After deadlock_timeout, one of these will receive:
-- ERROR: deadlock detected
```

Expected: One session gets the deadlock error and its transaction is rolled back. The other completes successfully.

Exercise 2 — Deadlock-Proof Transfer Function

```
-- Write a transfer function that is immune to deadlocks via consistent ordering
CREATE OR REPLACE FUNCTION safe_transfer(
    p_from_id bigint,
    p_to_id   bigint,
    p_amount  numeric
) RETURNS void AS $$
BEGIN
  IF p_from_id = p_to_id THEN
    RAISE EXCEPTION 'Cannot transfer to same account';
  END IF;

  -- Lock in consistent order (lower ID first) to prevent deadlock
```

```
  IF p_from_id < p_to_id THEN
    PERFORM id FROM accounts WHERE id = p_from_id  FOR UPDATE;
    PERFORM id FROM accounts WHERE id = p_to_id    FOR UPDATE;
  ELSE
    PERFORM id FROM accounts WHERE id = p_to_id    FOR UPDATE;
    PERFORM id FROM accounts WHERE id = p_from_id  FOR UPDATE;
  END IF;

  UPDATE accounts SET balance = balance - p_amount WHERE id = p_from_id;
  UPDATE accounts SET balance = balance + p_amount WHERE id = p_to_id;
END;
$$ LANGUAGE plpgsql;

-- Test with concurrent transfers in opposite directions
-- Session A: SELECT safe_transfer(1, 2, 100);
-- Session B: SELECT safe_transfer(2, 1, 50);
-- Neither should deadlock.
```

Exercise 3 — Monitor Deadlock Counts

```
-- Baseline
SELECT deadlocks FROM pg_stat_database WHERE datname = current_database();

-- Deliberately cause deadlocks (run sessions in parallel)
-- ... reproduce Exercise 1 multiple times ...

-- Check count increased
SELECT deadlocks FROM pg_stat_database WHERE datname = current_database();

-- Enable lock wait logging and reproduce
ALTER SYSTEM SET log_lock_waits = on;
SELECT pg_reload_conf();
-- Check PostgreSQL log files for lock wait entries
```

Production Troubleshooting Scenarios

Scenario 1: Sudden Deadlock Spike After Code Deploy

Symptom: `pg_stat_database.deadlocks` jumps from near-zero to hundreds per hour after a new release.

Diagnosis:

```
-- Enable lock wait logging immediately
ALTER SYSTEM SET log_lock_waits = on;
SELECT pg_reload_conf();

-- Check which tables are involved
SELECT relation::regclass, count(*)
FROM pg_locks
WHERE NOT granted
```

```
  AND locktype = 'relation'
GROUP BY relation::regclass
ORDER BY count(*) DESC;
```

Root Cause Pattern: New code introduced a second update path that locks tables/rows in opposite order from existing code. Look for the new code's UPDATE/DELETE order.

Fix: Audit all code paths that modify the same set of tables and enforce consistent ordering.

Scenario 2: Deadlocks Under Bulk Data Import

Symptom: Nightly batch job importing 5M rows has deadlock errors and re-runs.

Diagnosis:

```
-- Check if batch is doing row-by-row updates in random order
SELECT query, calls, mean_exec_time
FROM pg_stat_statements
WHERE query ILIKE '%UPDATE%import_table%'
ORDER BY calls DESC;
```

Fix:

```
-- Instead of row-by-row in random order:
LOOP
  UPDATE import_table SET status = 'processed' WHERE id = random_id;
END LOOP;

-- Do a bulk update in primary key order:
UPDATE import_table
SET status = 'processed'
WHERE id IN (SELECT id FROM import_table WHERE status = 'pending' ORDER BY id);
```

Scenario 3: Intermittent Deadlocks in Microservices

Symptom: Deadlocks occur only under load, between two different microservices.

Root Cause: Service A updates `orders` then `payments`. Service B updates `payments` then `orders` (for a different business operation that happens to touch the same rows).

Fix:

1. Identify which rows can be touched by both services.
2. Agree on a canonical lock-acquisition order (e.g., always `orders` then `payments`).
3. In Service B, reorder the updates to match Service A's order.
4. Add deadlock retry logic to both services as a safety net.

Cross-References

- `04_locks.md` — Lock modes, pg_locks, and lock monitoring fundamentals
- `06_optimistic_vs_pessimistic.md` — Alternative to pessimistic locking that avoids deadlocks

- `07_serializable_snapshot_isolation.md` — SSI resolves conflicts without traditional locks, but has its own abort/retry semantics
- `01_transaction_basics.md` — Transaction lifecycle; why ROLLBACK is needed after 40P01
- `../10_PostgreSQL_Internals/03_lock_manager.md` — Wait-for graph implementation details